

Реализация ETLпроцесса

В основу решения положена конфиденциальная информация.

Задание 1. Работа с Yandex DataTransfer

Цель: перенести данные из Managed Service for YDB в Object Storage с использованием сервиса Yandex Data Transfer.

Что было сделано:

1. Создание базы данных Yandex Managed Service for YDB.
2. Настроены права доступа для сервисного аккаунта, который в дальнейшем использовался во всех заданиях
3. Подготовка данных. Был написан генератор синтетических данных `gen_transactions.py`.
 - a. Генератор формирует CSV-файл с полями: `call_id`, `call_time`, `client_id`, `region_code`, `campaign_type`, `call_status`, `client_response`, `duration_sec`, `follow_up_required`.
4. Загрузка данных в YDB через Data Transfer. Вставка данных в YDB была выполнена не через YQL-запросы (INSERT), а через дополнительный трансфер:
 - a. Сгенерированный CSV-файл загружен в Yandex Object Storage.
 - b. Создан эндпоинт источник типа Object Storage с указанием формата CSV, схемы колонок и разделителя.
 - c. Создан эндпоинт приёмник типа YDB с указанием таблицы `transactions_v2`.
 - d. Создан и активирован трансфер типа Snapshot — данные были перенесены из Object Storage в YDB автоматически, без написания YQL INSERT-скриптов.
5. Создание основного трансфера: YDB → Object Storage.
 - a. Создан второй эндпоинт источник — YDB (с таблицей `transactions_v2`).
 - b. Создан второй эндпоинт приёмник — Object Storage (целевой бакет, формат выгрузки — CSV).
 - c. Создан и активирован трансфер типа Snapshot, который выгрузил содержимое таблицы YDB обратно в Object Storage.

Артефакты приложены в виде скринов выполненного задания, а также скриптов, которым создавалась таблица и генерировались данные.

Задание 2. Автоматизация работы с Yandex Data Processing при помощи Apache Airflow

Цель: автоматизировать обработку файлов через Yandex Data Processing (Hadoop + Spark), управляемого из Managed Service for Apache Airflow.

Что было сделано:

1. Подготовка инфраструктуры.
 - a. Был создан бакет, с которым будем работать в рамках задания. Внутри бакета есть разделение на несколько директорий - одна, в которой лежит script PySpark задания, а вторая с Dag файлом
 - b. Развёрнут Managed Service for Apache Airflow, к нему привязан сервисный аккаунт с правами на управление Data Processing
2. При запуске dag файла был создан кластер Yandex Data Processing, в рамках которого запускался файл с PySpark заданием на генерацию данных. Сама по себе генерация заняла достаточно много времени, однако получилось сгенерировать файл с общим размером в > 50 МБ. Далее последним заданием Dag файла - удаляем ранее созданный кластер

В рамках артефактов также приложены скриншоты по выполненным заданиям

Задание 3. Работа с топиками Apache Kafka с помощью PySpark в Yandex Data Processing

Цель: организовать потоковую обработку сообщений из Apache Kafka средствами Spark Structured Streaming, развернутого в Yandex Data Processing.

Что было сделано:

1. Подготовка архитектуры.
 - a. Развернут Yandex Managed Service for Apache Kafka (кластер из 1 брокера для экономии ресурсов)
 - b. Создан топик dataproc-kafka-topic с 1 партицией и replication factor = 1
 - c. Создан пользователь Kafka с правами Producer/Consumer
 - d. Развёрнут кластер Yandex Data Processing с компонентами HDFS, LIVY, SPARK, TEZ, YARN
 - e. Подготовлен бакет Object Storage для скриптов и результата.
2. Producer.
 - a. Написан Python скрипт kafka-write.py, который генерирует JSON сообщения, соответствующие требуемой структуре и отправляет сообщения в топик loanapplications с использованием библиотеки kafka python
 - i. суммарный объем отправленных данных — 25 МБ (несколько десятков тысяч сообщений)

3. PySpark задание потокового чтения и батчевого чтения
4. После сложил скрипты в объектное хранилище и запускал задания из кластера YDP для записи и потом чтения записанных данных в топик Kafka

Артефакты приложены, как скринами, так и файлами, которыми записывали/считывали данные из топика Kafka